

## Security Engineering

### 5. Übung

#### Aufgabe 1 (Password-Cracking (Dictionary-Attack))

Schreiben Sie ein einfaches Password-Cracking-Tool.

Die Wörterbuch-Attacke (Dictionary-Attack) bildet den verschlüsselten (oder gehashten) `passwd`-Eintrag für alle Wörter des Wörterbuchs. Wird dabei eine Übereinstimmung gefunden, so ist das Klartextpassword bekannt.

Beispiel: bei einem bekannten `passwd`-Eintrag von

```
$1$ZYXWVUTS$xN7BR1VyRwgiKy.Gfnss0/
```

würde die Wörterbuch-Attacke bei Benutzung der Wörter aus `/usr/share/dict/words` diese testen, z.B.

```
A      $1$ZYXWVUTS$y6t.HbZrqRxPSI5PT1eoW1
```

```
...
```

```
secpa  $1$ZYXWVUTS$Px290T6yKgy4P4C511yAd0
```

```
secque $1$ZYXWVUTS$2h2opR3NHAm9NUqEGmHbE/
```

```
secre  $1$ZYXWVUTS$3kghTnCpC90405tH2RInH1
```

```
secrecy $1$ZYXWVUTS$oiE5.nXUXQ/sF.Y.M5P1I0
```

```
secret $1$ZYXWVUTS$xN7BR1VyRwgiKy.Gfnss0/
```

und bei `secret` merken, dass der gesuchte Wert gefunden wurde, somit auf das Passwort `secret` schließen.

Schreiben Sie ein Shellskript, das ein Kommando der Form

```
openssl passwd -1 -salt ...
```

benutzt, um die Passwörter folgender Benutzer zu finden. Diese stammen alle aus einer UNIX `/usr/share/dict/words` Datei, siehe

<https://www-crypto.htwsaar.de/weber/download/words-f87a5a595d.txt> :

```
Oliver.Baumann $1$5D8i5QCj$K/mdUGFnTXCM6GCQszSQo0
```

```
Manuel.Neuer $1$wzDacj2a$whuyKxdot6UKnQfjbKIZ4/
```

```
Alexander.Nuebel $1$42XD/U40$Rgvxgy5AMMj/57N1EDEr0/
```

```
Waldemar.Anton $1$/mnsP843$VxG6fh48oIQIVKi0jYHuY/
```

Nathaniel.Brown \$1\$EkYR.T1L\$lrRMSLkI6KtZsH/y3lhBT.  
 Pascal.Gross \$1\$Wf06y5pF\$NyJJzY1Sh8iZwhE/LVjr00  
 Joshua.Kimmich \$1\$Gp7YxksQ\$j.IQUuS0fregY2Pu50Zh//  
 Felix.Nmecha \$1\$sIYuv/IK\$T1mfK7as0iToghgXjX/eM/  
 Aleksandar.Pavlovic \$1\$YU774r87\$BA1ue0cvHXCHoCgnWTuQc0  
 David.Raum \$1\$Dpq41Ny9\$Iuv9Mw.gGbg2nxtVRRAir0  
 Antonio.Ruediger \$1\$kQL6AWvi\$pkac5I0174BMi.u55SU4/.  
 Nico.Schlotterbeck \$1\$f0eL1Jbm\$n3Bed2SZXP4xXVoibKWV/1  
 Angelo.Stiller \$1\$p58ja.9x\$FLVaFQTjk8UL6uy9jKsXy.  
 Jonathan.Tah \$1\$mGL.oih0\$tKN4Ec/KqDMazDHSKCrhI1  
 Malick.Thiaw \$1\$WVIVB6iy\$wsDu1hfn3v1S7G8D8mWxW1  
 Nadiem.Amiri \$1\$7T8f4d4b\$8y4wz9qe.H4IHWXDLNSaJ.  
 Maximilian.Beier \$1\$UJi0eg2S\$XUCEW/rA7K1SacpYTznSw1  
 Leon.Goretzka \$1\$A/RMLHV7\$ToUqxiktkdwGARM13HLwm1  
 Kai.Havertz \$1\$pQKhPpn4\$Pm28TaBjKT1i7iCjSZLRp/  
 Assan.Ouedraogo \$1\$dubuT.Wa\$c34ppsazTle/NnZNC3hyf/  
 Jamie.Leweling \$1\$ni7AAwgm\$UyB0.sZyt2D7r5khrFcHr.  
 Jamal.Musiala \$1\$weK.VmxQ\$RxZ/juzxXX4nwIR63yikL0  
 Leroy.Sane \$1\$/nHZlDum\$AddmkCfM1.TV3iVcL0d3D1  
 Deniz.Undav \$1\$DHxUpHY9\$DfEHYFCB68MbxCY.RvS5l.  
 Florian.Wirtz \$1\$a00i0ued\$z1vs2MrFGWcw2W9Qp/Kqn0  
 Nick.Woltemade \$1\$ngbj/VuZ\$Fd7Vhdo/VfJ4.wsH43LZi/

## Aufgabe 2 (ONE-TIME-Passwörter / TAN-Listen)

Erzeugen Sie für einen gegebenen Username  $u$  und einer Anzahl  $n$  mit einem Shell-Skript `generate_tan` eine TAN-Liste von  $n$  Nummern für  $u$ , die für ein Online-Banking-System benutzt werden könnten. Verwenden Sie hierbei eines der in der Vorlesung vorgestellten Verfahren (Lamport, HOTP oder TOTP).

Simulieren Sie den Server durch ein Shell-Skript `bank_transaction`, das

- in einer Endlosschleife läuft,
- nicht mit [STRG+C] abgebrochen werden kann und
- von der Tastatur zwei Eingaben entgegennimmt
  - Username,
  - TAN

Pro User soll in einem Unterverzeichnis TAN eine Datei existieren, die die aktuelle TAN-Liste enthält. Überlegen Sie sich einen geeigneten Mechanismus, dass die aktuelle TAN geprüft wird.

Das Skript `bank_transaction` soll ausgeben

- Zugriff erlaubt, wenn die aktuelle TAN richtig war
- Zugriff verweigert, wenn
  - die aktuelle TAN falsch war oder

- die TAN-Liste des Users aufgebraucht ist

### Aufgabe 3 (Limits und Signale)

Sinn dieser Aufgabe ist die Beobachtung von Prozessen, die vorgegebene Limits überschreiten.

Setzen Sie vor Ausführung eines Prozesses folgende Limits

- cpu time
- stack size
- file size

und sorgen Sie dafür, dass der Prozess diese Limits überschreitet.

Statten Sie den Prozess mit einem Signalhandler aus, der das vom Betriebssystem generierte Signal abfängt und sich danach beendet.